

Universidade Estadual de Santa Cruz (UESC)

Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória compartilhada com OpenMP

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: Gabriel Oliveira Lucas César; Ronaldo Ribeiro Porto Filho

Disciplina: DEC107 — Processamento Paralelo

Data: 22/09/2026

1. Introdução

Este relatório apresenta a Etapa 1 (OpenMP) do Projeto 1, cujo objetivo é implementar o cálculo do Conjunto de Mandelbrot pelo método de *escape-time* em uma versão sequencial e em uma versão paralela com OpenMP, avaliando o impacto do número de threads, da política de escalonamento e do tamanho do *chunk* sobre o desempenho e o balanceamento de carga.

Para cada pixel da imagem, um ponto complexo c é obtido por mapeamento linear da região do plano complexo. A partir de $z_0 = 0$, aplica-se a recorrência

$$z_{n+1} = z_n^2 + c$$

até que $|z|^2 > 4$ ou seja atingido `MAX_ITER`. O número de iterações executadas é armazenado na matriz de contagens e representa também uma aproximação direta do trabalho computacional daquele pixel.

O problema é embaraçosamente paralelo porque cada pixel pode ser calculado de forma independente. Entretanto, o custo de cada pixel é irregular: alguns pontos escapam após poucas iterações, enquanto outros exigem centenas ou milhares. Dessa forma, dividir a imagem em regiões geometricamente iguais não significa necessariamente dividir o trabalho de forma equilibrada.

Na região padrão, 50% das linhas centrais concentram aproximadamente 97% do trabalho. No caso de *zoom* no vale dos cavalos-marinhos, o custo por pixel também é heterogêneo, porém a distribuição do trabalho entre as linhas é menos concentrada. Por esse motivo, o balanceamento de carga e as políticas de escalonamento são elementos centrais desta etapa.

Em sistemas de memória compartilhada, várias threads executam utilizando o mesmo espaço de memória, e o OpenMP permite expressar esse paralelismo por meio de diretivas inseridas no código. As políticas de escalonamento definem como as iterações de um laço são distribuídas entre as threads: `static` faz essa divisão previamente, `dynamic` entrega novos blocos conforme as threads terminam seu trabalho e `guided` segue uma ideia semelhante, começando com blocos maiores e reduzindo seu tamanho ao longo da execução. A análise de desempenho é importante nesse contexto porque permite avaliar não apenas o ganho obtido com mais threads, mas também os custos do paralelismo e a qualidade da distribuição do trabalho.

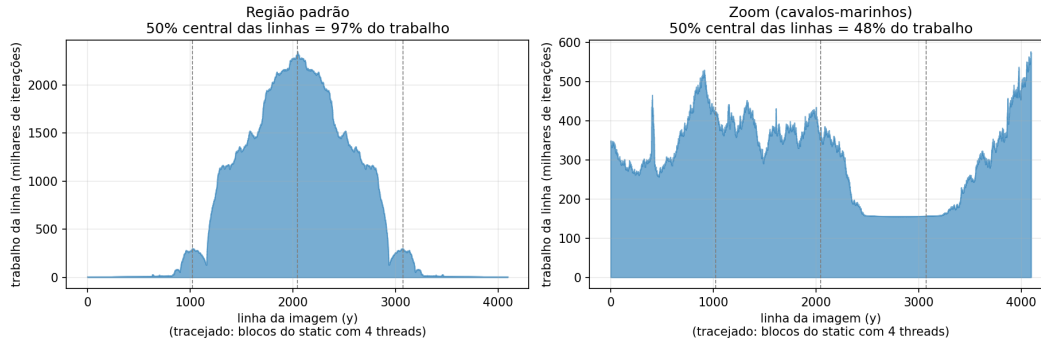


Figura 1 — Trabalho agregado por linha na região padrão e no zoom. As linhas tracejadas ilustram os blocos do *static* sem *chunk* para quatro *threads*.

2. Metodologia

2.1 Implementação sequencial

A implementação sequencial percorre as linhas e colunas da imagem, mapeando cada pixel para o plano complexo e calculando o *escape-time* em precisão *double*.

Pseudocódigo simplificado:

```
para y = 0 até N-1:
    ci = mapear_imaginario(y)

    para x = 0 até N-1:
        cr = mapear_real(x)
        zr = 0
        zi = 0
        contagem = MAX_ITER

        para i = 0 até MAX_ITER-1:
            zr_novo = zr*zr - zi*zi + cr
            zi_novo = 2*zr*zi + ci

            se zr_novo*zr_novo + zi_novo*zi_novo > 4:
                contagem = i + 1
                interromper

            zr = zr_novo
            zi = zi_novo

        matriz[y][x] = contagem
```

A matriz de saída é armazenada em inteiros de 32 bits e gravada em formato binário (*.bin*), usado como saída canônica para a validação de corretude. Uma imagem PPM também pode ser gerada para inspeção visual.

A região padrão utiliza $\text{Re} \in [-2,0; 1,0]$, $\text{Im} \in [-1,5; 1,5]$, resolução 4096×4096 e $\text{MAX_ITER} = 1000$. O caso de zoom utiliza centro $(-0,743643887; 0,131825904)$, largura real de $3,0 \times 10^{-3}$ e $\text{MAX_ITER} = 5000$.

Para a validação contra a referência fornecida pelo professor, a matriz produzida pelo código C sequencial foi comparada, pixel a pixel, com a saída da função `mandelbrot()` do arquivo `gera_mandelbrot.py` disponibilizado pelo professor. A referência foi executada com os mesmos parâmetros dos casos padrão e de zoom. Durante essa comparação identificou-se uma diferença de convenção na contagem do escape: a referência registra o índice `i`, iniciado em zero, enquanto nossa implementação registra `i + 1`, isto é, o número de iterações efetivamente executadas. Para isolar as demais diferenças, essa convenção foi compensada apenas na análise comparativa, sem modificar os binários originais nem o código utilizado nos benchmarks.

2.2 Paralelização com OpenMP

A paralelização foi aplicada ao laço externo, que percorre as linhas da imagem. Essa escolha mantém uma granularidade mais grossa que a distribuição por pixel, mas ainda disponibiliza 4096 iterações independentes para no máximo 12 threads.

Trecho simplificado:

```
#pragma omp parallel default(none) shared(img2D, tempoThread, trabalhoThread, nLin, nCol)
{
    int tid = omp_get_thread_num();
    long long trabalhoLocal = 0;

    #pragma omp barrier
    double inicio = omp_get_wtime();

    #pragma omp for schedule(runtime) nowait
    for (int y = 0; y < nLin; y++) {
        /* calcula a linha y */
    }

    tempoThread[tid] = omp_get_wtime() - inicio;
    trabalhoThread[tid] = trabalhoLocal;
}
```

As principais decisões foram:

Elemento	Justificativa
<code>parallel + for</code>	Distribui as linhas entre as threads, mantendo cada pixel independente
<code>schedule(runtime)</code>	Permite alterar <code>static</code> , <code>dynamic</code> , <code>guided</code> e o <code>chunk</code> sem recompilar
<code>default(none)</code>	Obriga classificação explícita das variáveis compartilhadas
<code>nowait</code>	Permite registrar o instante em que cada thread termina o laço, necessário ao cálculo de <code>F_LB</code>
<code>barrier</code> antes do cronômetro	Alinha o início da medição das threads

Elemento	Justificativa
Sem critical , atomic ou reduction	Cada thread escreve em posições distintas e acumula o trabalho localmente
Sem collapse	O laço externo já fornece paralelismo suficiente para o hardware utilizado

Como cada thread escreve em posições distintas da matriz e acumula seu trabalho localmente, não são necessárias operações de sincronização para a atualização dos pixels. Possíveis efeitos de compartilhamento de cache não foram medidos separadamente e não constituíram o foco desta etapa.

2.3 Configurações experimentais

Foram testadas as políticas **static**, **dynamic** e **guided**. Nos testes específicos de política $\times$ *chunk*, realizados com 6 e 12 threads, foram avaliados **static** em blocos e com *chunks* 1, 4, 16, 64 e 256; **dynamic** com *chunks* 1, 4, 16, 64 e 256; e **guided** com *chunks* 1, 4, 16 e 64. A escalabilidade forte foi avaliada com 2, 4, 6, 8 e 12 threads.

No *weak scaling*, manteve-se aproximadamente constante a quantidade de pixels por thread:

Threads	Resolução
1	4096×4096
2	5793^2
4	8192^2
6	10033^2
8	11585^2

O ponto de 12 threads no *weak scaling* não foi executado por uma limitação preventiva adotada no script para o alvo de 32 bits, evitando resoluções maiores que 11585^2 e o risco de ultrapassar o espaço de endereçamento disponível.

2.4 Ambiente e procedimento de medição

Os experimentos foram executados em:

Item	Valor
CPU	Intel Core i5-12400F
Núcleos físicos / threads lógicas	6 / 12
Memória	32 GB
Sistema operacional	Windows 11
Compilador	GCC 6.3.0 (MinGW.org), alvo <code>mingw32</code>
Flags	<code>-O2 -msse2 -mfpmath=sse -fopenmp -lm</code>
Cronômetro	<code>omp_get_wtime()</code>
Plano de energia	Equilibrado
Condições	OneDrive pausado; navegadores/Discord fechados; computador sem outras tarefas durante os testes

As mesmas flags de compilação foram utilizadas nas versões sequencial e OpenMP para manter o ambiente experimental consistente e facilitar a reprodução dos resultados.

Apenas a região de cálculo foi cronometrada; alocação e escrita em disco ficaram fora da medição principal. O tempo de escrita do arquivo binário foi medido separadamente: na resolução base, a mediana foi de aproximadamente 0,042 s na versão sequencial e 0,041 s na versão OpenMP. Cada configuração OpenMP foi executada 5 vezes e os resultados são apresentados pela mediana. As configurações foram embaralhadas entre as repetições para reduzir o efeito de variações térmicas ou de clock.

Foram registradas 505 execuções no total, sendo 480 OpenMP e 25 sequenciais. A resolução observada do cronômetro foi de aproximadamente 1 ms; por isso, valores muito pequenos de F_{LB} devem ser interpretados considerando essa limitação experimental.

As métricas utilizadas foram:

$$S(p) = T_{seq} / T_p$$

$$E(p) = S(p) / p$$

$$F_{LB} = (T_{max} - T_{min}) / T_{max}$$

onde T_{max} e T_{min} são, respectivamente, o maior e o menor tempo de laço entre as threads.

3. Resultados

3.1 Corretude

A corretude interna foi verificada comparando os arquivos binários produzidos pelas versões sequencial e OpenMP. Em cada execução, o SHA-256 da matriz paralela foi comparado ao da nossa versão sequencial do mesmo caso e resolução.

Nas 505 execuções realizadas:

- 505 registros apresentaram `hash_ok = True`;
- nenhuma configuração executada divergiu da configuração solicitada;
- não houve falhas de execução.

Nos dois casos oficiais preservados em `output/ref`, a comparação direta entre os binários sequencial e OpenMP resultou em zero pixels diferentes. Portanto, as versões sequencial e OpenMP desenvolvidas produziram exatamente as mesmas matrizes.

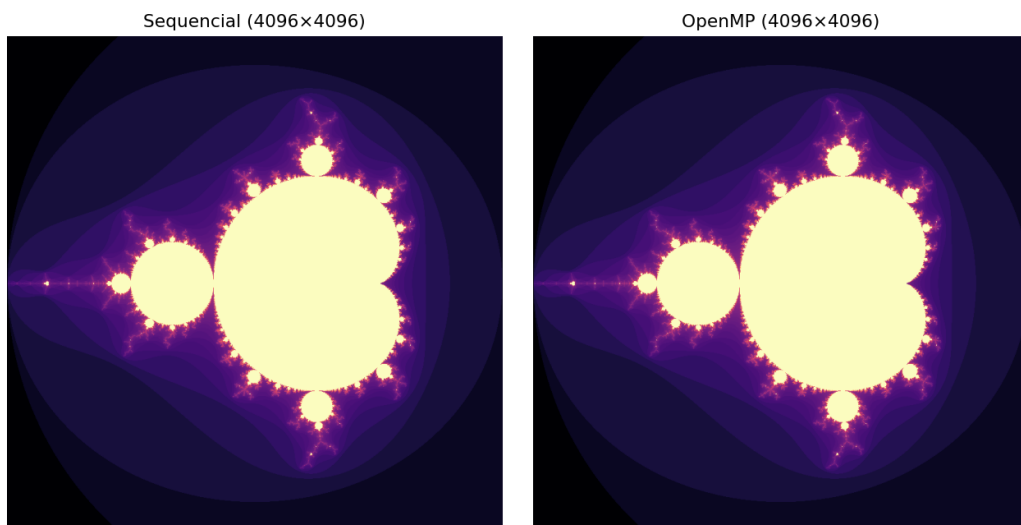


Figura 2 — Resultado visual da versão sequencial e da versão OpenMP para a região padrão. A validação efetiva é feita sobre a matriz binária.

Também foi realizada uma comparação com a função `mandelbrot()` disponibilizada pelo professor em `gera_mandelbrot.py`. Nessa comparação foi identificada uma diferença de convenção: a referência registra `i`, iniciado em zero, enquanto nossa implementação registra `i + 1` para os pixels que escapam. Para comparar as duas convenções, esse deslocamento foi compensado apenas durante a validação, sem alterar os códigos ou os arquivos binários utilizados nos benchmarks.

Após essa compensação, permaneceram as divergências abaixo:

Caso	Pixels divergentes	Percentual da imagem	Maior diferença
Padrão	786	0,004685%	460 iterações
Zoom	24.242	0,144494%	1.415 iterações

Assim, a comparação com a referência do professor não atingiu integralmente o limite de tolerância indicado no enunciado: no caso padrão, o percentual ficou abaixo de 0,01%, mas existem diferenças superiores a uma iteração; no zoom, o percentual de pixels divergentes também ficou acima de 0,01%. Esses valores são apresentados de forma explícita no relatório e podem ser reproduzidos pelo script `codigos_auxiliares/validar_professor.py`.

Independentemente dessa comparação externa, a equivalência entre as versões sequencial e OpenMP desenvolvidas foi exata nos dois casos oficiais.

3.2 Região padrão — tempo, Speedup e Eficiência

Para a região padrão, o baseline sequencial foi de aproximadamente 6,98 s.

Com `dynamic,1`, os resultados foram:

Threads	Tempo (s)	Speedup	Eficiência
1	6,987	1,00×	100%
2	3,596	1,94×	97%

Threads	Tempo (s)	Speedup	Eficiência
4	1,885	$3,70\times$	93%
6	1,289	$5,41\times$	90%
8	0,980	$7,12\times$	89%
12	0,709	$9,84\times$	82%

Com uma thread, a versão OpenMP apresentou tempo praticamente igual ao sequencial, indicando que não foi observado overhead relevante da infraestrutura OpenMP para esse tamanho de problema.

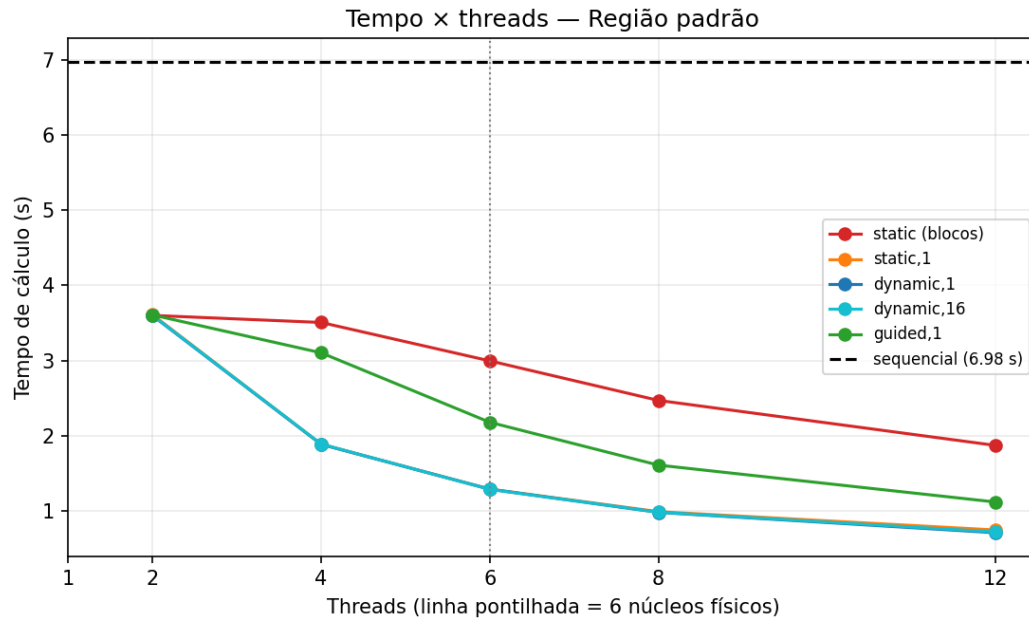


Figura 3 — Tempo de cálculo em função do número de threads na região padrão.

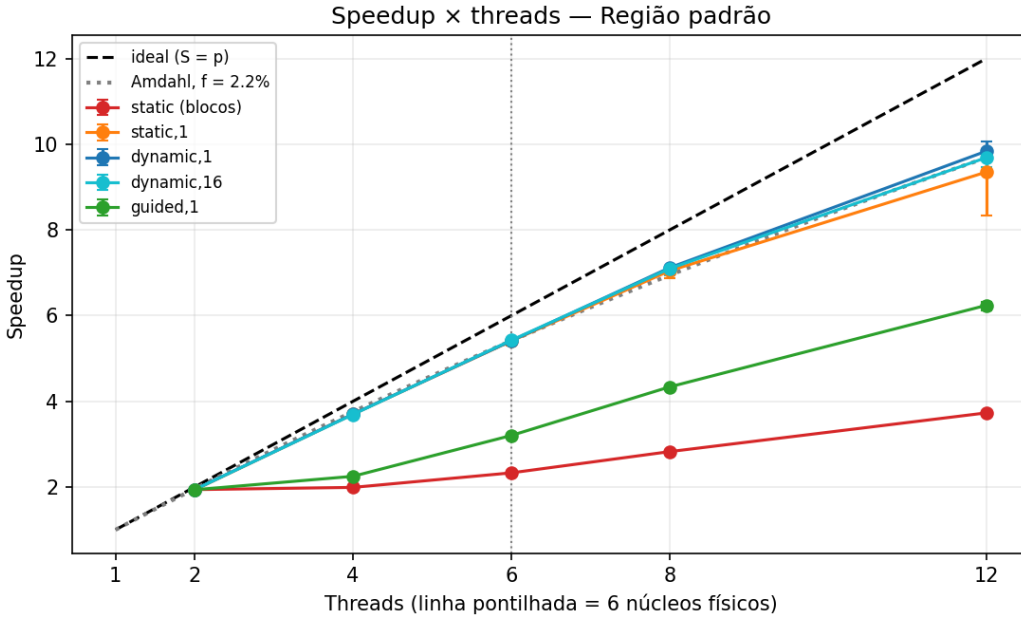


Figura 4 — Speedup em função do número de threads para as políticas avaliadas.

A Figura 4 também apresenta uma curva de referência baseada na Lei de Amdahl, usando uma estimativa da fração do tempo que não escala com o número de threads, calculada a partir do Speedup medido. Esse valor resume, em um único número, os efeitos que limitam o ganho observado e não deve ser interpretado como uma medição direta da porcentagem de código sequencial.

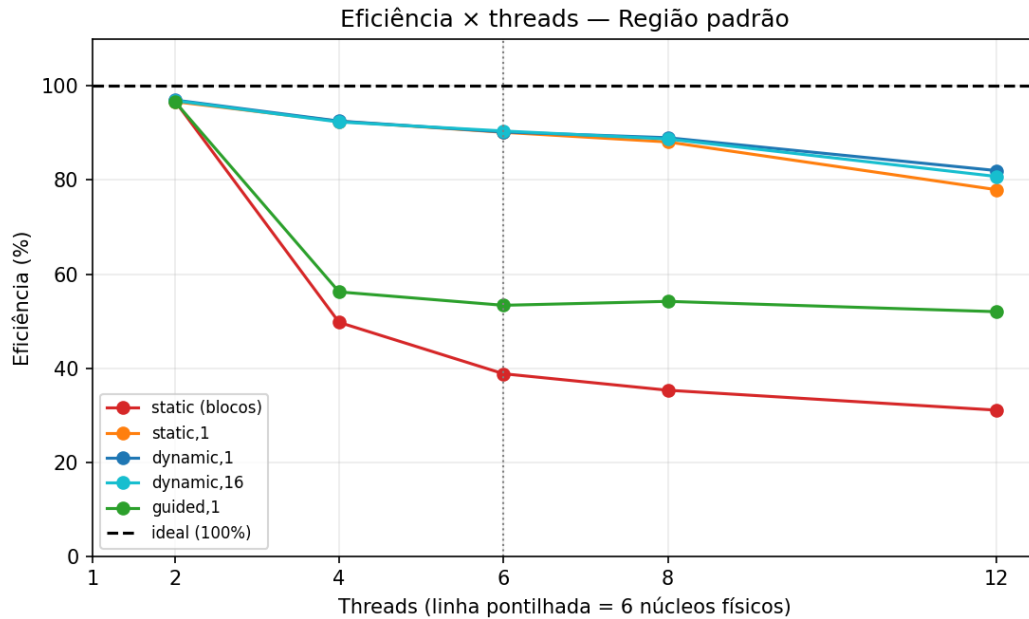


Figura 5 — Eficiência em função do número de threads. A linha vertical marca os 6 núcleos físicos.

Não foi observada saturação até as 12 threads lógicas disponíveis no processador. Configurações acima desse valor implicariam *oversubscription* e não foram avaliadas nesta etapa.

3.3 Políticas de escalonamento e tamanho do chunk

Com 12 threads na região padrão, `dynamic,1`, `dynamic,4` e `dynamic,16` apresentaram tempos muito próximos. Embora `dynamic,4` tenha obtido a menor mediana nominal (0,701 s), a diferença para `dynamic,1` (0,709 s) e `dynamic,16` (0,720 s) está dentro da dispersão observada; portanto, não se conclui que `dynamic,4` seja superior.

O resultado principal é que *chunks* pequenos preservaram bom balanceamento e baixo tempo, enquanto *chunks* maiores aumentaram progressivamente o desbalanceamento.

O caso mais desfavorável na região padrão foi o `static` sem *chunk*, que distribui blocos contíguos de linhas. Como o trabalho está concentrado nas linhas centrais, essa estratégia produziu `F_LB` próximo de 1 e desempenho muito inferior às políticas mais equilibradas.

3.4 Zoom — balanceamento e efeito do chunk

No zoom, o baseline sequencial foi de aproximadamente 3,10 s. Com `dynamic,1` e 12 threads, o tempo caiu para aproximadamente 0,323 s, correspondendo a Speedup de $9,59\times$ e Eficiência próxima de 80%.

O comportamento das políticas foi diferente do observado na região padrão. Como o custo por linha é menos concentrado, o `static` em blocos sofreu menos, embora continuasse inferior às melhores configurações dinâmicas.

Política	Tempo (s)	Speedup	Eficiência (%)	F_LB
static (blocos)	0,443	6,99	58	0,650
static,1	0,339	9,14	76	0,104
dynamic,1	0,323	9,59	80	0,006
dynamic,16	0,340	9,11	76	0,063
guided,1	0,357	8,68	72	0,149

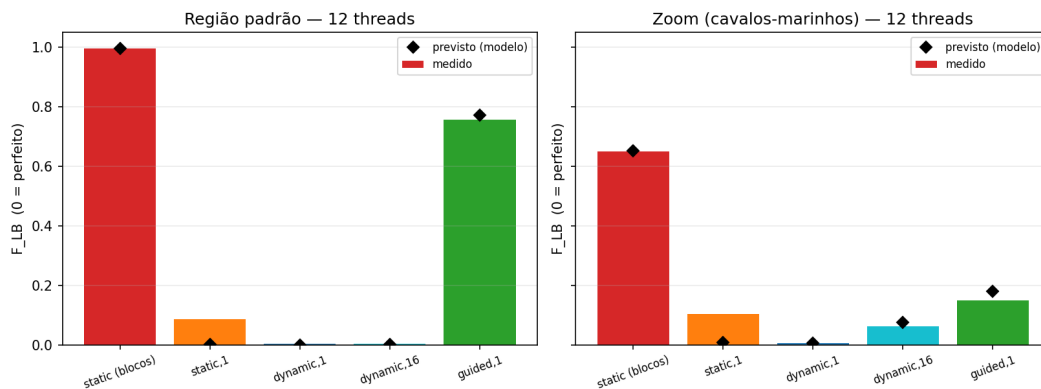


Figura 6 — Fator de Balanceamento de Carga medido e previsto para diferentes políticas, na região padrão e no zoom, com 12 threads.

O tamanho do *chunk* mostrou um compromisso claro entre granularidade e balanceamento. Em `dynamic`, valores pequenos mantiveram `F_LB` baixo, enquanto *chunks* maiores aumentaram progressivamente o desbalanceamento.

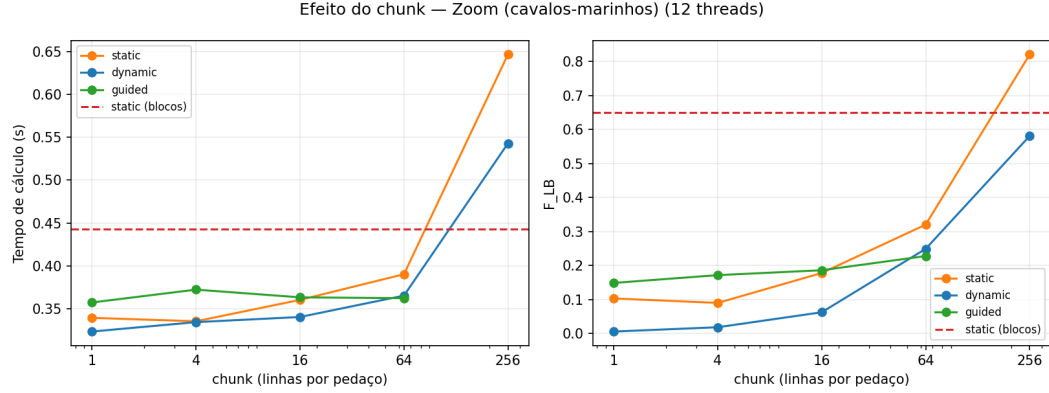


Figura 7 — Efeito do tamanho do chunk sobre o tempo de cálculo e o Fator de Balanceamento de Carga no zoom.

3.5 Escalabilidade fraca

No *weak scaling*, a quantidade de pixels por thread foi mantida aproximadamente constante. O tempo ideal, portanto, seria constante.

Os tempos observados cresceram de aproximadamente 6,99 s com 1 thread para 7,85 s com 8 threads, mantendo eficiência fraca próxima de 89% no maior caso testado.

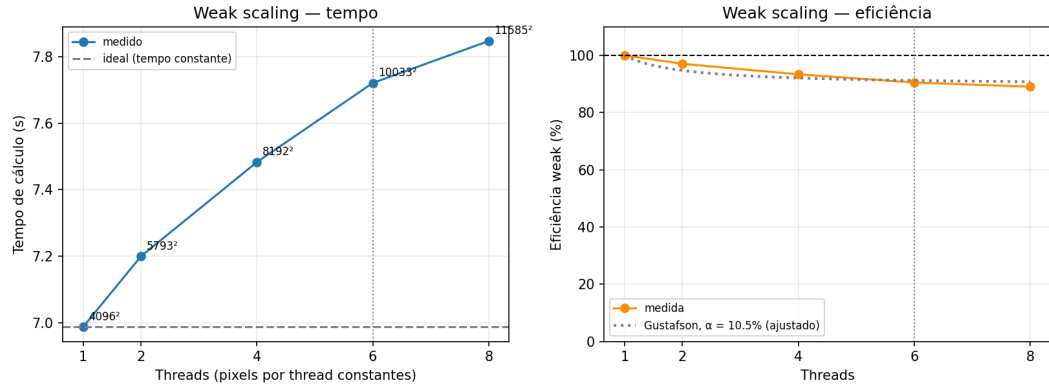


Figura 8 — Weak scaling: tempo de cálculo e eficiência à medida que o problema cresce proporcionalmente ao número de threads.

O comportamento é coerente com a interpretação de Gustafson: ao aumentar proporcionalmente o tamanho do problema e os recursos computacionais, o tempo permanece relativamente estável, embora efeitos de hardware e overhead reduzam gradualmente a eficiência.

4. Discussão

Os resultados mostram que a paralelização do Mandelbrot com OpenMP é efetiva, mas que o ganho de desempenho depende não apenas do número de threads, e sim de como o trabalho é distribuído entre elas. Como cada pixel é independente, a implementação possui alto potencial de paralelismo.

Entretanto, o número de iterações executadas por pixel varia fortemente de acordo com sua posição no plano complexo, tornando o balanceamento de carga um fator determinante.

4.1 Corretude e comparação com a referência

As versões sequencial e OpenMP desenvolvidas produziram matrizes idênticas entre si nos dois casos oficiais, o que confirma que a paralelização preservou o resultado da implementação sequencial.

Na validação contra a referência fornecida pelo professor, foi necessário primeiro considerar a diferença de convenção de contagem (i na referência e $i + 1$ em nossa implementação). Mesmo após essa compensação apenas para fins de comparação, permaneceram 786 pixels divergentes no caso padrão (0,004685%) e 24.242 no zoom (0,144494%).

Esses valores não atendem integralmente ao limite de tolerância indicado no enunciado e, por esse motivo, são apresentados de forma explícita no trabalho. A validação foi automatizada pelo script `validar_professor.py`, que gera um relatório próprio sem modificar as matrizes originais utilizadas nos benchmarks.

4.2 Impacto do número de threads

Na região padrão, o uso de `dynamic, 1` reduziu o tempo de aproximadamente 6,98 s na versão sequencial para 0,709 s com 12 threads, resultando em Speedup de 9,84 e Eficiência de aproximadamente 82%. O ganho foi próximo do ideal até os núcleos físicos disponíveis e continuou crescendo com o uso das threads lógicas.

A eficiência diminuiu gradualmente à medida que o número de threads aumentou. Esse comportamento é compatível com os custos adicionais do paralelismo e com o compartilhamento de recursos do processador, embora esses efeitos não tenham sido medidos separadamente. Assim, o Speedup não cresce de forma perfeitamente linear.

Não foi observada saturação até as 12 threads lógicas disponíveis. Configurações acima desse valor não foram avaliadas, pois representariam *oversubscription*, isto é, mais threads de software que contextos de execução fornecidos pelo hardware.

4.3 Balanceamento de carga e políticas de escalonamento

O principal efeito observado está relacionado à distribuição espacial do custo computacional. Na região padrão, as linhas centrais concentram a maior parte das iterações. Consequentemente, o `static` sem `chunk`, que distribui blocos contíguos de linhas, atribui quantidades semelhantes de linhas às threads, mas quantidades muito diferentes de trabalho. Isso explica o alto F_{LB} e o desempenho inferior dessa configuração.

A distribuição `static` com `chunk` pequeno reduz esse problema porque espalha ciclicamente linhas de regiões diferentes entre as threads. Já o `dynamic` redistribui novos blocos durante a execução, permitindo que threads que terminam regiões leves recebam trabalho adicional. Nos experimentos, essa estratégia produziu os menores valores de F_{LB} e os menores tempos entre as políticas avaliadas.

O `guided` apresentou comportamento intermediário. Como inicia a execução distribuindo blocos maiores e reduz seu tamanho progressivamente, parte do desbalanceamento espacial pode ser definida ainda nas primeiras distribuições. Isso ajuda a explicar por que ele não alcançou o mesmo equilíbrio observado no `dynamic` com `chunks` pequenos.

Os resultados também mostram que não existe vantagem em reduzir ou aumentar o *chunk* indefinidamente. Valores pequenos aumentam a flexibilidade do escalonamento, mas também elevam o número de solicitações ao *runtime*. Valores grandes reduzem esse overhead, porém diminuem a capacidade de redistribuir regiões caras. Com *chunk*=256, por exemplo, as 4096 linhas da imagem formam apenas 16 blocos para 12 threads, deixando pouca margem para compensar diferenças de custo entre eles. O melhor comportamento foi obtido com *chunks* pequenos, e as diferenças entre `dynamic,1`, `dynamic,4` e `dynamic,16` ficaram dentro da dispersão experimental, não sendo adequado afirmar superioridade estatística de uma delas.

4.4 Região padrão e zoom

Embora o zoom tenha sido utilizado como caso de desbalanceamento acentuado, as medições mostraram que os dois casos apresentam distribuições de custo distintas. Na região padrão, o trabalho por linha é fortemente concentrado nas linhas centrais, tornando o `static` em blocos particularmente desfavorável. No zoom, o custo por pixel permanece altamente variável, mas o trabalho agregado por linha é menos concentrado.

Essa diferença explica por que o `static` em blocos apresentou F_{LB} de aproximadamente 0,996 na região padrão e 0,650 no zoom. Portanto, o comportamento de uma política de escalonamento não depende apenas do valor de `MAX_ITER`, mas da distribuição espacial efetiva do custo no domínio processado.

Esse resultado reforça a importância de caracterizar a carga antes de escolher uma política. Uma divisão geometricamente uniforme da imagem não implica necessariamente uma divisão uniforme de trabalho.

4.5 Escalabilidade fraca e gargalos

No teste de *weak scaling*, a quantidade de pixels por thread foi mantida aproximadamente constante. O tempo cresceu de cerca de 6,99 s com uma thread para 7,85 s com oito threads, mantendo eficiência próxima de 89% no maior caso executado. O comportamento é compatível com a interpretação de Gustafson: aumentando simultaneamente o tamanho do problema e os recursos computacionais, é possível manter o tempo de execução relativamente estável.

A perda gradual de eficiência é compatível com custos adicionais do paralelismo e com o compartilhamento de recursos do processador. Como esses fatores não foram medidos separadamente, os experimentos permitem observar o efeito global sobre o tempo de execução, mas não atribuir uma parcela específica da perda a cada causa.

Outro limite experimental foi o uso de um toolchain com alvo de 32 bits. Por precaução, o script não executou o ponto de 12 threads no *weak scaling*, evitando uma resolução acima do limite preventivo adotado para essa coleta. A resolução observada de aproximadamente 1 ms do cronômetro também limita a interpretação de valores de F_{LB} muito próximos de zero.

4.6 Limitações e possíveis melhorias

A implementação atende ao objetivo desta etapa e demonstra ganhos expressivos com OpenMP. Como trabalhos futuros, seria possível utilizar um toolchain de 64 bits para ampliar os testes de *weak scaling* e investigar outras formas de particionamento da imagem.

Os resultados desta etapa já mostram que uma estratégia simples de paralelismo por linhas, combinada com escalonamento adequado, foi suficiente para obter Speedup próximo de 10 no hardware utilizado.

5. Conclusão

Este trabalho apresentou a implementação sequencial e paralela, com OpenMP, do cálculo do conjunto de Mandelbrot pelo método de *escape-time*. Os experimentos mostraram que o problema possui elevado potencial de paralelização, uma vez que cada pixel pode ser processado de forma independente, mas também evidenciaram que a irregularidade do custo computacional torna o balanceamento de carga um aspecto central do desempenho.

Na região padrão, a versão sequencial apresentou tempo mediano de aproximadamente 6,98 s. Com 12 threads e `dynamic,1`, esse tempo foi reduzido para aproximadamente 0,709 s, correspondendo a Speedup de 9,84 e Eficiência de aproximadamente 82%. No caso de zoom, o tempo caiu de aproximadamente 3,10 s para 0,323 s, resultando em Speedup de 9,59 e Eficiência próxima de 80%. Esses resultados mostram que a implementação OpenMP conseguiu explorar tanto os 6 núcleos físicos quanto as 12 threads lógicas disponíveis no processador.

A análise das políticas de escalonamento mostrou que dividir a imagem em quantidades iguais de linhas não implica necessariamente distribuir quantidades iguais de trabalho. Na região padrão, as linhas centrais concentram aproximadamente 97% das iterações, fazendo com que o `static` em blocos contíguos apresente forte desbalanceamento. Políticas com *chunks* pequenos, especialmente as configurações dinâmicas, distribuíram melhor as regiões de maior custo. Entretanto, também foi observado que o menor fator de desbalanceamento não implica obrigatoriamente o menor tempo de execução, pois o gerenciamento dinâmico do trabalho introduz overhead. Dessa forma, a escolha do escalonamento representa um compromisso entre distribuição da carga e custo de gerenciamento.

Os testes de *weak scaling* também apresentaram comportamento satisfatório. Mantendo aproximadamente constante a quantidade de pixels por thread, a eficiência permaneceu próxima de 89% no maior caso executado, com 8 threads. Isso indica que o aumento do tamanho do problema pode ser acompanhado pelo aumento dos recursos computacionais sem um crescimento proporcional do tempo de execução, embora efeitos de hardware, sincronização e escalonamento reduzam gradualmente a eficiência.

Quanto à corretude, as versões sequencial e OpenMP desenvolvidas produziram resultados idênticos entre si nos dois casos oficiais. Na comparação adicional com a referência do professor, após compensar apenas a diferença de convenção i versus $i + 1$, permaneceram divergências de 0,004685% dos pixels no caso padrão e 0,144494% no zoom. Esses valores não atendem integralmente ao limite de tolerância indicado no enunciado e são registrados explicitamente neste relatório. Ainda assim, a paralelização preservou exatamente o resultado produzido pela implementação sequencial desenvolvida.

A comparação conceitual com o DGEMM reforça um dos principais aprendizados da etapa. Ambos os problemas apresentam unidades de trabalho independentes e, portanto, permitem paralelismo de dados. Entretanto, o DGEMM possui carga computacional regular e previsível: para matrizes de dimensões fixas, cada elemento da matriz resultante exige essencialmente a mesma quantidade de multiplicações e somas. Por isso, uma divisão estática em linhas ou blocos de tamanhos semelhantes tende a distribuir o trabalho de forma equilibrada e com baixo overhead de escalonamento.

No Mandelbrot, por outro lado, o custo de cada pixel depende do número de iterações necessárias até

o escape, podendo variar de poucas iterações até `MAX_ITER`. Como pixels de custo semelhante também se concentram espacialmente, regiões geometricamente iguais podem representar quantidades muito diferentes de trabalho. Assim, enquanto no DGEMM a divisão do trabalho tende a ser mais regular, no Mandelbrot o balanceamento de carga assume papel mais relevante. A principal conclusão dessa comparação é que disponibilizar mais threads não é suficiente para garantir bom desempenho: a estratégia de paralelização deve considerar a distribuição real do custo computacional do algoritmo.

6. Referências

1. UESC. *DEC107 — Processamento Paralelo: materiais das Aulas 03, 04, 06, 07, 08 e 09*. Universidade Estadual de Santa Cruz, 2026.
 2. UESC. *Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): enunciado do projeto*. DEC107 — Processamento Paralelo, 2026.
 3. OPENMP ARCHITECTURE REVIEW BOARD. *OpenMP Application Programming Interface Specification*. Disponível em: <https://www.openmp.org/specifications/>. Acesso em: 22 set. 2026.
 4. AMDAHL, Gene M. *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. AFIPS Conference Proceedings, 1967.
 5. SUN, Xian-He; CHEN, Yong. *Reevaluating Amdahl's Law in the Multicore Era*. Journal of Parallel and Distributed Computing, 2010.
 6. VORELLANA, E. T. *meuGEMM: teste de desempenho baseado na rotina DGEMM*. Repositório utilizado como material de apoio da disciplina DEC107 — Processamento Paralelo.
 7. VORELLANA, E. T. *RepoDEC107-PP-2026_2*. Repositório da disciplina DEC107 — Processamento Paralelo, 2026.
-

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Nós, Gabriel Oliveira Lucas César e Ronaldo Ribeiro Porto Filho, regularmente matriculados no curso de Ciência da Computação, declaramos, para os devidos fins, que, na elaboração do trabalho intitulado “Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot (Escape-Time): Implementação sequencial e paralela em memória compartilhada com OpenMP”, submetido como requisito da disciplina DEC107 — Processamento Paralelo, adotamos as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa.

(Assinale a opção aplicável)

☐ Não utilizamos ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

☒ Utilizamos ferramentas de Inteligência Artificial (ex.: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram **ChatGPT e Claude**. O uso ocorreu exclusivamente nas seguintes etapas do projeto:

- revisão, explicação e depuração de trechos específicos das implementações em C e OpenMP;
- apoio na estruturação e revisão dos scripts auxiliares utilizados na automação dos experimentos, consolidação dos resultados e geração de gráficos;
- auxílio na interpretação dos resultados experimentais, incluindo Speedup, Eficiência, balanceamento de carga, políticas de escalonamento e efeito do tamanho do *chunk*;
- apoio na investigação e documentação das diferenças numéricas entre a implementação desenvolvida e a versão serial de referência fornecida pelo professor;
- organização, revisão técnica, gramatical e estrutural do relatório, README e demais documentos do projeto;
- apoio na organização da comparação conceitual entre a paralelização do Mandelbrot e o caso de estudo do DGEMM apresentado na disciplina.

Termo de Responsabilidade: Declaramos estar cientes de que ferramentas de Inteligência Artificial não são isentas de erros, vieses ou alucinações. Sendo assim, atestamos que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente nossa. Confirmamos que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por nós, garantindo que os resultados refletem nosso próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus – BA, 22 de setembro de 2026.

Gabriel Oliveira Lucas César
Matrícula: 202410123

Ronaldo Ribeiro Porto Filho
Matrícula: 202410131